2520030123

POLISETTY SREEHARSHA

CASE STUDY 4: HOTEL RESERVATIONS AND DATE RANGES

Q1. Domain Invariants and Value Objects

1.1 Business Invariants

Domain invariants are rules that must always be satisfied in a system to maintain data consistency and prevent invalid operations.

1. Valid Check-in and Check-out Dates

- The check-in date must always be before the check-out date.
- A reservation must cover at least one night.
- Check-out on the same date as check-in or before check-in is not allowed.

2. Room Availability

- A room cannot be assigned to two different reservations for overlapping stay periods.
- A room can have multiple reservations only when their stay periods do not overlap.
- Consecutive reservations are allowed when one guest checks out on the same date another guest checks in.

3. Reservation Confirmation

- A reservation can only be confirmed if the requested room is available for the entire stay period.
- A reservation must have a valid payment guarantee before it can be confirmed.
- An unconfirmed reservation cannot be treated as a confirmed booking.

4. Payment Guarantee

- Every confirmed reservation must have a valid payment guarantee.
- A payment guarantee may be provided through a credit card, advance payment, or another approved payment method.
- If the payment guarantee is invalid or missing, the reservation must not be confirmed.

5. Cancellation Policy

- A reservation can only be cancelled before the guest checks in.
- Cancellation after check-in is not permitted through the normal reservation cancellation process.
- Refund amounts must follow the hotel's cancellation policy.
- Full refunds must not be provided for cancellations made after check-in.

6. Guest Check-in and Check-out Operations

- A guest can check in only if the reservation is confirmed.
- A guest cannot check in before the reservation's check-in date.
- A guest can check out only after successfully checking in.
- A reservation cannot be checked out more than once.

7. Reservation Status Management

- A reservation must follow a valid sequence of states, such as:

PENDING → CONFIRMED → CHECKED_IN → CHECKED_OUT

- A cancelled reservation cannot be checked in. A checked-out reservation cannot be cancelled.

1.2 Value Objects and Their Justification

- Value objects are domain objects that are identified by their values rather than by a unique identity. They are generally immutable and do not have their own lifecycle.

Value Object	Justification
StayPeriod (DateRange)	Represents check-in and check-out dates. Its validity depends on the dates, not an identity.
RoomIdentifier	Represents a unique room number or code. It is immutable and identifies a room through its value.
ReservationIdentifier	Represents a unique reservation reference. It is immutable and used to identify a reservation.
PaymentGuarantee	Represents payment authorization or guarantee details. Its value determines whether the guarantee is valid.
CancellationPolicy	Represents cancellation rules, deadlines and refund conditions. It can be immutable and compared based on its values.

Business invariants ensure that invalid reservations, overlapping bookings, and incorrect cancellations are prevented. Value objects help maintain data integrity by representing important domain information as immutable objects.

Q2. Designing DateRange and Reservation

2.1 Introduction

In a hotel reservation system, a customer's stay period is represented using two dates:

- **Check-in Date:** The date on which the guest arrives at the hotel.
- **Check-out Date:** The date on which the guest leaves the hotel.

A DateRange or StayPeriod class is used to represent this period.

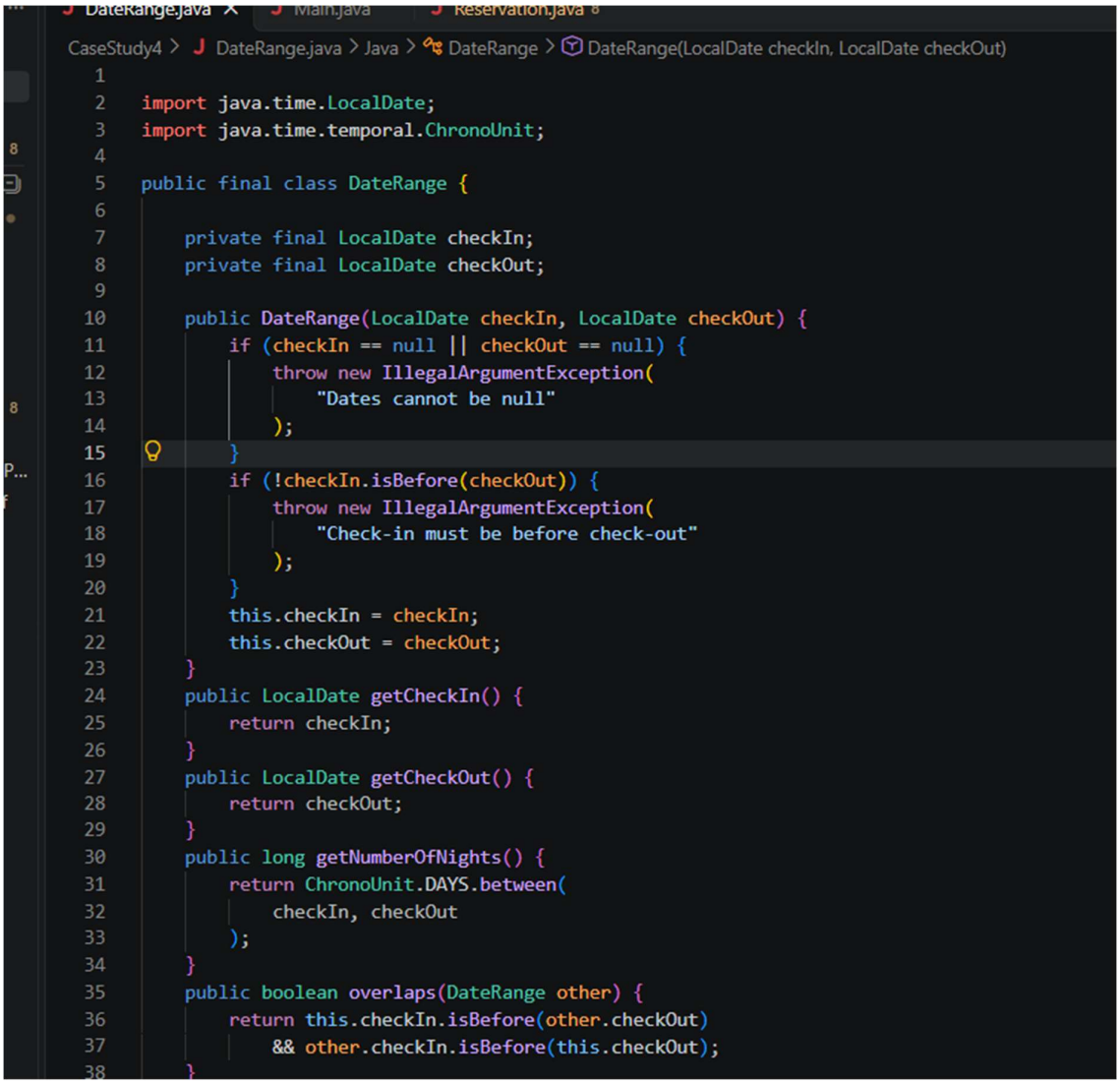
The design must ensure that invalid date ranges cannot be created and that two reservations cannot overlap for the same room.

2.2 Java Implementation of DateRange

We use Java's LocalDate class from the java.time package to represent dates.

The DateRange class contains:

- Check-in date.
- Check-out date.
- Constructor validation.
- A method to calculate the number of nights.
- A method to check whether two stay periods overlap.



2.3 Explanation of the Java Methods

1. Constructor Validation

```
if (!checkIn.isBefore(checkOut)) {
   throw new IllegalArgumentException(
      "Check-in must be before check-out"
   );
}
```

This condition ensures that the check-in date is always earlier than the check-out date.

2.3 Overlapping Date Range Method

```
public boolean overlaps(DateRange other) {
```

```
  return this.checkIn.isBefore(other.checkOut)
     && other.checkIn.isBefore(this.checkOut);
}
```

This method checks whether two stay periods overlap.

2.4 Example of Overlapping and Consecutive Reservations

Example 1: Overlapping Reservations

Reservation A

10 Nov – 13 Nov

3 nights

10

11

12

Reservation B

12 Nov – 15 Nov

3 nights

12

13

14

Overlapping: 12 November

The same room cannot be assigned to both reservations.

Example 2: Consecutive Reservations

Reservation A: 10 Nov – 13 Nov

10

11

12

13

Reservation B: 13 Nov – 16 Nov

13

14

15

No overlap – Both reservations are allowed.

13 November is the checkout date of A and the check-in date of B. It is not counted as an occupied night in either stay.

The overlap method returns:

- true for overlapping reservations.
- false for consecutive reservations.

2.5 Reservation Class Design

The Reservation class represents a customer's booking and connects the guest, room and stay period.

DateRange.java | Main.java | Reservation.java 8

CaseStudy4 > Reservation.java > Java > Reservation

```java

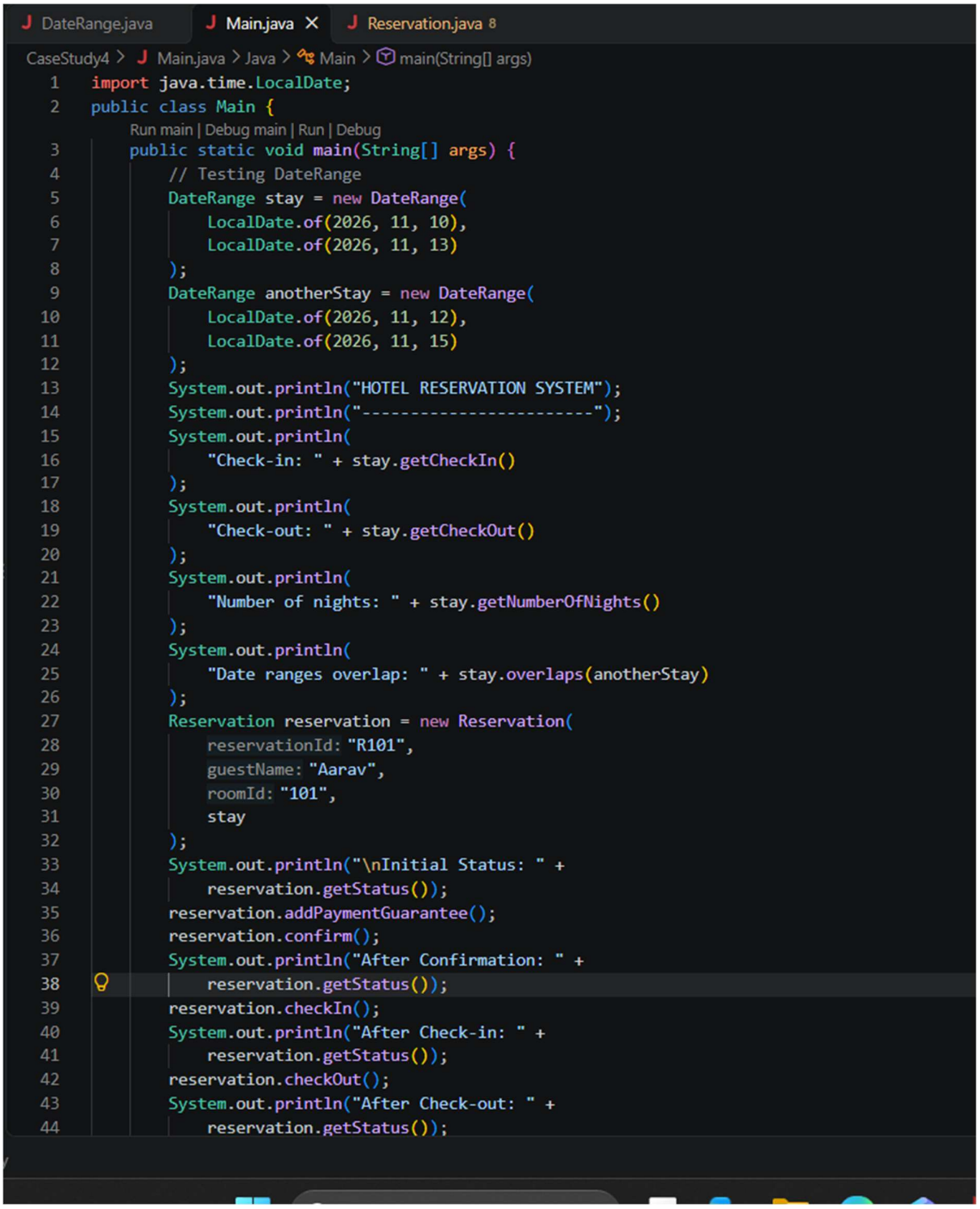
public class Reservation {
    private final String reservationId;
    private final String guestName;
    private final String roomId;
    private final DateRange stayPeriod;

    private String status;
    private boolean paymentGuaranteed;
    public Reservation(
        String reservationId,
        String guestName,
        String roomId,
        DateRange stayPeriod
    ) {
        this.reservationId = reservationId;
        this.guestName = guestName;
        this.roomId = roomId;
        this.stayPeriod = stayPeriod;
        this.status = "PENDING";
        this.paymentGuaranteed = false;
    }
    public void confirm() {
        if (!paymentGuaranteed) {
            throw new IllegalStateException(
                "Payment guarantee is required"
            );
        }
        if (!status.equals("PENDING")) {
            throw new IllegalStateException(
                "Invalid reservation status"
            );
        }

        status = "CONFIRMED";
    }
    public void addPaymentGuarantee() {
        paymentGuaranteed = true;
    }
    public void checkIn() {
        if (!status.equals("CONFIRMED")) {
            throw new IllegalStateException(
                "Reservation is not confirmed"
            );
        }
```


DateRange.java
Main.java
Reservation.java 8
CaseStudy4 > Reservation.java > Java > Reservation
2 public class Reservation {
23 public void confirm() {
32);
33 }
34
35 status = "CONFIRMED";
36 }
37 public void addPaymentGuarantee() {
38 paymentGuaranteed = true;
39 }
40 public void checkIn() {
41 if (!status.equals("CONFIRMED")) {
42 throw new IllegalStateException(
43 "Reservation is not confirmed"
44);
45 }
46 status = "CHECKED_IN";
47 }
48 public void checkOut() {
49 if (!status.equals("CHECKED_IN")) {
50 throw new IllegalStateException(
51 "Guest has not checked in"
52);
53 }
54 status = "CHECKED_OUT";
55 }
56 public void cancel() {
57 if (!status.equals("PENDING")
58 && !status.equals("CONFIRMED")) {
59 throw new IllegalStateException(
60 "Cancellation is not allowed"
61);
62 }
63 status = "CANCELLED";
64 }
65 public String getStatus() {
66 return status;
67 }
68 }
69

Main.java:


DateRange.java
Main.java
Reservation.java 8
CaseStudy4 > Main.java > Java > Main > main(String[] args)
import java.time.LocalDate;
public class Main {
Run main | Debug main | Run | Debug
public static void main(String[] args) {
// Testing DateRange
DateRange stay = new DateRange(
LocalDate.of(2026, 11, 10),
LocalDate.of(2026, 11, 13)
);
DateRange anotherStay = new DateRange(
LocalDate.of(2026, 11, 12),
LocalDate.of(2026, 11, 15)
);
System.out.println("HOTEL RESERVATION SYSTEM");
System.out.println("------------------------");
System.out.println(
"Check-in: " + stay.getCheckIn()
);
System.out.println(
"Check-out: " + stay.getCheckOut()
);
System.out.println(
"Number of nights: " + stay.getNumberOfNights()
);
System.out.println(
"Date ranges overlap: " + stay.overlaps(anotherStay)
);
Reservation reservation = new Reservation(
reservationId: "R101",
guestName: "Aarav",
roomId: "101",
stay
);
System.out.println("\nInitial Status: " +
reservation.getStatus());
reservation.addPaymentGuarantee();
reservation.confirm();
System.out.println("After Confirmation: " +
reservation.getStatus());
reservation.checkIn();
System.out.println("After Check-in: " +
reservation.getStatus());
reservation.checkOut();
System.out.println("After Check-out: " +
reservation.getStatus());

Output:



2.6 How the Design Prevents Invalid Date Ranges

The design prevents invalid date ranges in the following ways:

1. The constructor checks that both dates are not null.
2. It verifies that the check-in date is earlier than the check-out date.

3. The class uses final fields, so dates cannot be changed after object creation.
4. The constructor throws an exception if an invalid date range is provided.
5. The overlap method detects conflicting reservations.

Q3. Entity Classification and Aggregate Design

3.1 Classification of Domain Concepts

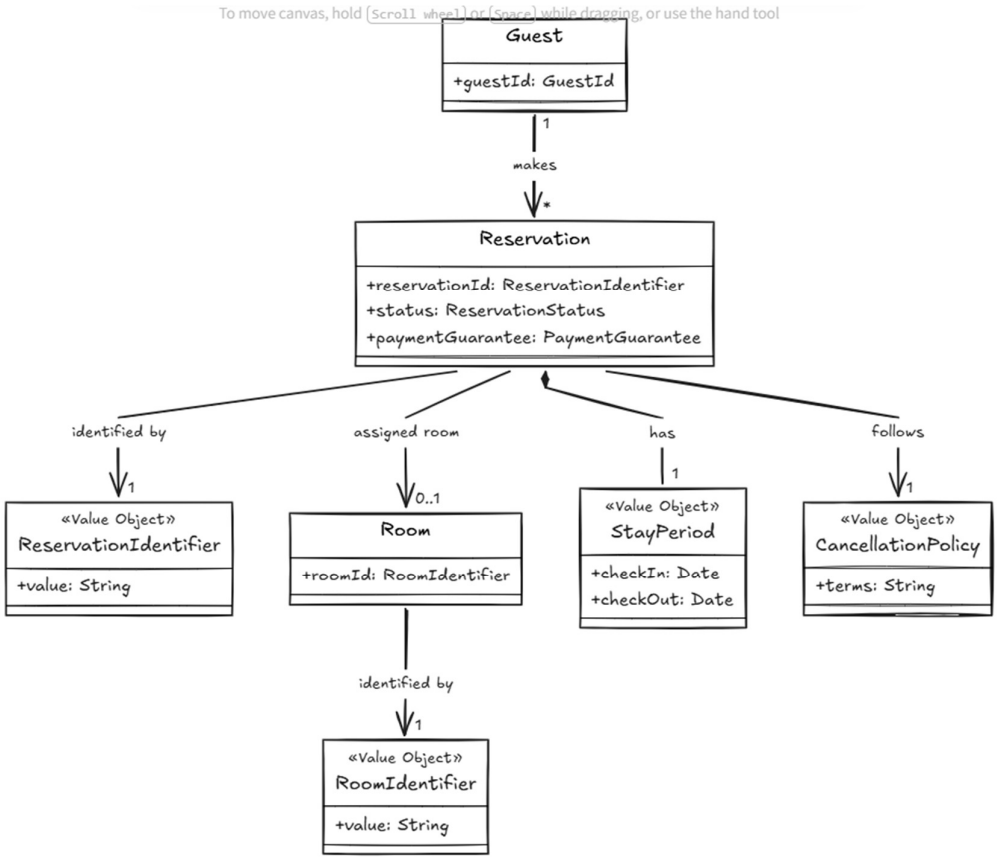
In Domain-Driven Design (DDD), domain concepts are classified as either entities or value objects.

- **Entities:** Objects that have a unique identity and maintain their identity throughout their lifecycle.
- **Value Objects:** Objects that are identified by their attributes or values rather than by a unique identity

Domain Concept	Classification	Justification
Guest	Entity	A guest has a unique guest ID and maintains identity across multiple reservations.
Room	Entity	A room has a unique room ID and exists independently of reservations.
Reservation	Entity	A reservation has a unique reservation ID and changes its status throughout its lifecycle.
Room Identifier	Value Object	It represents the identity of a room through an immutable value.
Reservation Identifier	Value Object	It uniquely identifies a reservation and does not have an independent lifecycle.
Stay Period	Value Object	It represents check-in and check-out dates and is identified by its date values.
Cancellation Policy	Value Object	It represents cancellation deadlines, rules and refund conditions.

3.2 UML Class Diagram

The following UML diagram represents the relationships between the hotel reservation domain concepts.



1. A Guest can make multiple Reservations.
2. Each Reservation belongs to one Guest.
3. Each Reservation is assigned to one Room.
4. Each Reservation contains one StayPeriod.
5. Each Reservation is identified by a ReservationIdentifier.
6. Each Room is identified by a RoomIdentifier.
7. Each Reservation follows a CancellationPolicy.

3.3 Appropriate Consistency Boundary for Room Availability

The appropriate consistency boundary for enforcing room availability can be a RoomSchedule or RoomInventory aggregate, which manages reservations for a room and ensures that overlapping bookings are not allowed.

An aggregate is a group of related domain objects that are treated as one consistency unit. An aggregate root controls access to the objects within it and ensures that business rules are maintained.

Proposed Design:

- Room acts as the aggregate root.
- Room availability is managed through the Room aggregate.
- Reservations associated with the room are checked before assigning or confirming a booking.
- The system verifies that no existing active reservation overlaps with the requested stay period.
- If an overlapping reservation exists, the new booking is rejected.

Q4. Reservation Lifecycle and Making Illegal States Unrepresentable

4.1 Reservation State Management

A reservation passes through different stages during its lifecycle.

The states are:

- Requested
- Confirmed
- Checked In
- Completed
- Cancelled

A suitable approach is to use a **State Machine with an Enum** in Java.

Java Code – Reservation State Enum

```
public enum ReservationStatus {
  REQUESTED,
  CONFIRMED,
  CHECKED_IN,
  COMPLETED,
  CANCELLED
}
```

4.2 Reservation Lifecycle

REQUESTED

Confirm with valid payment and room availability

CONFIRMED

Guest arrives and checks in

CHECKED IN

Guest checks out

COMPLETED

CANCELLED

Terminal state – reached from REQUESTED or CONFIRMED, subject to policy

4.3 Permitted and Prohibited Transitions

Current State	Next State	Permission
Requested	Confirmed	Allowed
Requested	Cancelled	Allowed
Confirmed	Checked In	Allowed
Confirmed	Cancelled	Allowed according to policy
Checked In	Completed	Allowed
Checked In	Cancelled	Prohibited
Completed	Cancelled	Prohibited
Cancelled	Confirmed	Prohibited
Completed	Confirmed	Prohibited

4.4 Information Required for Reservation Confirmation

Before confirming a reservation, the system must have:

1. Valid Guest ID and guest details.
2. Valid Room ID.
3. Valid check-in and check-out dates.

4. Room availability for the requested stay period.
5. A valid payment guarantee.

4.5 Preventing Cancellation After Check-in

Cancellation must be allowed only when the reservation is in REQUESTED or CONFIRMED state.

If the reservation is already CHECKED_IN or COMPLETED, the system must reject the cancellation request.

EX:

```
public void cancel() {
   if (status != ReservationStatus.REQUESTED &&
      status != ReservationStatus.CONFIRMED) {
      throw new IllegalStateException(
         "Cancellation is not allowed at this stage"
      );
   }
   status = ReservationStatus.CANCELLED;
}
```

This method prevents hotel staff from cancelling reservations after the guest has checked in.

4.6 Making Illegal States Unrepresentable

The following techniques can be used:

- Use Enum to define only valid reservation states.
- Keep the status field private.
- Allow state changes only through specific methods.
- Validate every state transition before updating the status.
- Make important reservation details immutable.

Q5. Concurrent Reservations and Cancellation Policies

5.1 Problem with Concurrent Reservations

When two hotel receptionists try to book the same room for overlapping dates at almost the same time, a **Race Condition** can occur.

For example:

- Receptionist A checks room availability and finds it available.
- Receptionist B also checks the same room and finds it available.
- Both receptionists confirm their bookings.

5.2 Protecting the Application Against Concurrent Reservations

The application can prevent this problem using the following techniques:

1. **Database Transactions:** Perform the availability check and reservation insertion as one transaction.
2. **Pessimistic Locking:** Lock the room record while processing a booking so that another receptionist must wait.
3. **Optimistic Locking:** Use a version number to detect whether another transaction has modified the room data.
4. **Overlap Validation:** Check existing reservations before accepting a new booking.

5.3 Refund Calculation During Cancellation

The refund amount should be calculated according to the hotel's cancellation policy.

The policy may consider:

- Time of cancellation.
- Whether the guest has already checked in.
- Cancellation deadline.
- Applicable cancellation charges.

$$Refund = Total\ Payment \times \frac{Refund\ Percentage}{100}$$

Example: If the customer paid ₹10,000 and is eligible for a 50% refund, the refund amount is ₹5,000.

Refunds after check-in must follow the applicable post-check-in policy and must not automatically be 100%.

5.4 Additional Persistence-Level Safeguards

The following safeguards should be implemented at the database level:

1. **Primary Key Constraint:** Every room and reservation must have a unique identifier.
2. **Foreign Key Constraint:** A reservation must reference an existing guest and room.
3. **Check Constraints:** Check-in date must be before check-out date.
4. **Unique Constraints:** Prevent duplicate reservation references and other duplicate records.
5. **Transaction Management:** Ensure booking operations are completed atomically.
6. **Overlap Prevention:** Use database locking or an exclusion constraint to prevent overlapping reservations for the same room.